



Chinmoy Sarkar <chinmoy6667@gmail.com>

Assignment Marks

1 message

Programming Hero <admin@programming-hero.com>
To: chinmoy6667@gmail.com

Wed, Jun 17, 2026 at 5:14 PM



17 June, 2026

Assignment Result



Your Score

30/30

Assignment-2

Examiner's Feedback

DevPulse-API-Endpoints

1. Auth: POST /api/auth/signup - Register new user with validation, hashing, and exact response format

- **Input validation: name, email (unique), password, role (contributor/maintainer)** →

Validation logic correct! ✓

- **Password hashed with bcrypt (salt rounds 8-12), never exposed in response** →

Password security implemented! ✓

- **Role defaults to 'contributor'; email uniqueness enforced** → Role defaulting and

uniqueness check correct! ✓

- **Exact success response format (201 Created) with id, name, email, role, timestamps** →

Response structure matches spec! ✓

→ Excellent! ✓ User registration endpoint fully compliant with spec.

2. Auth: POST /api/auth/login - Authenticate user, generate JWT with id/name/role payload, exact response

- **Validate credentials and compare password with bcrypt** → Credential validation correct! ✓

- **JWT payload includes user id, name, and role** → JWT payload structure correct! ✓

- **Token signed correctly with expiry; returned in exact response format** → Token

generation and response format correct! ✓

- **Response includes both 'token' and 'user' object with exact fields** → Full response structure matches spec! ✓

→ Perfect! ✓ Login flow with JWT implemented correctly.

3. Issues: POST /api/issues - Create issue with JWT auth, reporter_id from token, validation, exact 201 response

- **JWT authentication middleware applied; reject invalid/missing token** → Auth middleware working! ✓

- **reporter_id extracted from decoded JWT (req.user.id), NOT from request body** →

Reporter assignment from token correct! ✓

- **Validation: title (≤150 chars), description (≥20 chars), type (bug/feature_request), status defaults to 'open'** → Input validation logic perfect! ✓

- **Exact 201 Created response format with all required fields** → Response structure matches spec! ✓

→ Great work! ✓ Issue creation endpoint fully functional and secure.

4. Issues: GET /api/issues - Public list with sort/type/status filters, reporter assembled WITHOUT SQL JOINS

- **Public access (no auth required); correct default sort (newest)** → Access control and default sorting correct! ✓

- **Query params: sort (newest/oldest), type (bug/feature_request), status (open/in_progress/resolved)** → Filter/sort logic working! ✓

- **Reporter object assembled WITHOUT SQL JOINS (separate query or WHERE id IN (...))**

→ Reporter data assembled without JOINS! ✓ Critical requirement met!

- **Exact success response format with array of issues including reporter object** → Response structure matches spec! ✓

→ Excellent! ✓ Filtering, sorting, and reporter assembly without JOINS implemented correctly.

5. Issues: GET /api/issues/:id - Public single issue fetch with reporter data, proper 404 handling

- **Public access; return 404 for non-existent issue ID** → Access control and 404 handling correct! ✓

- **Reporter data assembled WITHOUT SQL JOINS (separate query)** → Reporter assembly without JOINS correct! ✓

- **Exact success response format with full issue object including reporter** → Response structure matches spec! ✓

- **Correct HTTP status codes: 200 for success, 404 for not found** → Status codes correct! ✓

→ Well done! ✓ Single issue retrieval with reporter assembly correct.

6. Issues: PATCH /api/issues/:id - Update with role-based permissions (Maintainer: any; Contributor: own+open only)

- **JWT authentication and role extraction from token** → Auth and role verification correct! ✓

- **Permission logic: Maintainer can update any; Contributor only own issue AND only if status==='open'** → Role-based permission logic perfect! ✓

- **Validation on updatable fields; updated_at auto-refreshed on change** → Validation and timestamp update correct! ✓

• **Exact 200 OK response format with updated issue data** → Response structure matches spec! ✓

→ Perfect! ✓ Permission logic and update flow implemented correctly.

7. Issues: DELETE /api/issues/:id - Maintainer-only deletion with proper 403/404 handling

• **JWT authentication; strict maintainer-only role verification** → Auth and role check correct! ✓

• **Proper error handling: 403 for insufficient role, 404 for non-existent issue** → Error status codes correct! ✓

• **Exact success response: 200 with {success, message} OR 204 No Content** → Response format matches spec! ✓

→ Great! ✓ Deletion endpoint with strict maintainer check implemented correctly.

8. Global: Standardized success/error response structures and correct HTTP status codes across all endpoints

• **All success responses follow {success: true, message, data} structure** → Success response structure consistent! ✓

• **All error responses follow {success: false, message, errors?} structure; passwords never exposed** → Error response structure and security correct! ✓

• **Correct HTTP status codes used: 200/201/400/401/403/404/409/500 as per spec** → HTTP status codes perfectly aligned with spec! ✓

→ Excellent! ✓ Consistent response patterns and error handling throughout the API.

DevPulse-Interview-Video

9. Interview: Answer any 1 question from the provided list (event loop / middleware / error handling / SQL vs NoSQL / connection pooling)

• **Technical Accuracy: Correct explanation of the chosen concept with relevant details** → Technical content accurate and thorough! ✓

• **Clarity & Communication: Speaks naturally in English, logical flow, within 3-5 minutes, face visible, landscape mode** → Delivery clear and professional! ✓

→ Excellent explanation! ✓ Technical concept demonstrated with clarity and depth.

10. Interview: Answer a second distinct question from the provided list with same quality standards

• **Technical Accuracy: Correct explanation of the second chosen concept** → Technical content accurate and well-explained! ✓

• **Clarity & Communication: Natural English delivery, logical structure, 3-5 min, landscape, well-lit, public link** → Professional delivery and video quality! ✓

→ Great job! ✓ Second answer demonstrates strong understanding and communication skills.

Examiner Feedback: Impressive! You've excelled in this assignment. Keep honing your skills for even greater accomplishments. Best wishes from Programming Hero Team.

Never expose your (.env). Read that again.

Important Instructions:

→ Do not post on Facebook, if you have any marks-related issues.

→ Make sure to read all the requirements carefully, If you have any marks-related confusion.

→ If you are confident and If there is a mistake from the examiner's end, give a recheck request.

→ If your recheck reason was not valid, 2 marks will be deducted from your current marks.

Let's Code_ Your Career

Team Programming Hero

Let's Code Your Career



Site: web.programming-hero.com | Phone: +8801322901105

Address: Level-4, House 1162, [Road 10](#), [Avenue 12](#), [Dhaka](#) 1216

